

GridWise — Three-Minute Demonstration Script

A complete, ready-to-read narration for the walkthrough video: eight timed beats with the exact words to say, what should be on screen, and where each differentiator lands. Everything is done in the browser using only the public link — no terminal, no editor.

Live URL	https://brian-sheffield-surely-fly.trycloudflare.com
Repository	https://github.com/meherabmehu/GridWise-LLM-Energy-Optimizer
Runtime	3:00 — 8 beats, 9 screen actions
Word budget	474 spoken words, about 165 words per minute

Before you press record

1. The tunnel window stays open, the laptop stays on the charger, and sleep is disabled. The link dies within seconds if any of those change.
2. Open <https://brian-sheffield-surely-fly.trycloudflare.com/health> in Chrome tab one and <https://brian-sheffield-surely-fly.trycloudflare.com/docs> in tab two, then confirm tab one shows `{"status":"ok"}`.
3. Chrome zoom 110–125 percent, bookmarks bar hidden, notifications off, personal tabs closed, and record the browser in full screen (F11).
4. Keep this PDF open in a third tab. You will paste five request bodies — BODY A to BODY E from the companion demo guide.
5. Rehearse once with a stopwatch, then record. Do not record the first run.

Pacing is the whole game. Read each beat at about 165 words per minute and pause for one full second after every Execute. If you finish before 2:50, slow down — do not add new content. If you overrun, cut the Docker sentence in beat 7 before cutting anything else.

The script

0:00 — 0:15 1. Open: what this is

ON SCREEN The /health page already open in Chrome

SAY: “Hi — this is GridWise, our entry for the BUP CSE Fest preliminary round. Two endpoints: a health probe, and one that turns plain-English operator notes into a cost-optimal twenty-four hour energy schedule.”

DO Start already standing on the health page. Do not scroll, do not touch the mouse.

DELIVERY Smile, breathe, speak slowly. This is the one sentence you must not rush — aim for about two and a half words per second.

0:15 – 0:45 2. How it works, in one breath

ON SCREEN Still on the health page

SAY: “Three ideas hold it together. A real language model reads the notes and returns structured directives, and nothing else — it never sees the schedule and never does arithmetic. Deterministic guardrails clamp that output, so the model cannot change demand, solar, tariffs or battery limits. A deterministic linear program then solves the schedule, and an independent validator replays it before anything is returned. The solver runs two passes: minimum cost, then minimum battery cycling at that same cost — so we never pay for useless charge-and-discharge churn.”

DO No mouse movement, no clicks. This is the technical heart of the video.

DELIVERY **Land the last sentence clearly.** The two-pass solver is a differentiator almost no other team will have implemented.

0:45 – 0:58 3. The two links a judge checks first

ON SCREEN health page, then switch tab to /docs

SAY: “The readiness probe returns status ok — that is the judge's health check. And /docs is live Swagger, so anyone can run a real request by hand, exactly as I am about to.”

DO Point at {"status": "ok"}, then switch to the second tab showing /docs and point at both endpoint rows.

DELIVERY Open /docs in its own tab before you press record.

0:58 – 1:25 4. Case 1 — official sample SAMPLE-01

ON SCREEN /docs → POST /optimize-energy → Try it out → paste BODY A → Execute

SAY: “Sample one has two notes. The panels get washed from noon to two PM, so usable solar drops to twenty-five percent in hours twelve and thirteen. The second note is about a registration deadline — nothing to do with energy. The interpretation reads solar reduction, factor zero-point-two-five, hours twelve and thirteen. And the unrelated note comes back as no-op — the model refuses to invent a rule where none exists. Totals: 2,692.5 kilowatt-hours, 38,365 BDT, peak 175 — the organiser's published optimum.”

DO Paste, click Execute, then rest the mouse on total_cost_bdt while you read the total out loud.

DELIVERY Pause one full second after Execute so the response is visible on camera. If a validation note appears under the box while pasting, ignore it and click Execute anyway.

1:25 – 1:50 5. Case 2 — official sample SAMPLE-05

ON SCREEN same page → paste BODY B → Execute

SAY: “Sample five caps grid import at 155 kilowatt-hours between six and nine PM. The interpreter resolves that to hours 18, 19 and 20 — start inclusive, end exclusive, exactly the convention used in the official data. The plan lands on 33,950 BDT with a 175 peak. Again, the published optimum.”

DO Point at the hours array, then at the total.

DELIVERY Saying “start inclusive, end exclusive” out loud tells the judge you read the specification properly. Most teams miss it.

1:50 – 2:25 6. Cases 3 to 5 — the generalisation proof

ON SCREEN paste BODY C, then BODY D, then BODY E

SAY: “Now the part that matters most. This note is not in the sample set and is worded completely differently: the feeder is limited, keep hourly grid import under 155. Different words, same directive — same hours, same cap. That is generalisation, not phrase matching. Next, a note with nothing to do with energy — it returns no-op, so no bogus constraint ever reaches the solver. And this one is brand new: do not charge the battery between two and five AM. It returns no-charge-window for hours two, three and four, dropping the end hour — exactly as the official samples do.”

DO Three separate pastes and Executes, one second of stillness after each.

DELIVERY **This is your highest-scoring segment.** The hidden tests are paraphrases, so proving generalisation live is worth more than re-showing official cases.

2:25 – 2:45 7. Case 6 — robustness, and Docker

ON SCREEN Type /optimize-energy and then /no-such-path in the address bar

SAY: “Bad input never breaks it. A GET on the POST endpoint gives 405, an unknown path gives 404, a malformed body gives 400 with a clean JSON error — no crash, no stack trace, never the API key. The project also ships as a Docker image: it builds, reports healthy, and answers with a live model call from inside the container.”

DO Show the two error pages briefly — two seconds each, no more.

DELIVERY Docker is described verbally because it is not on screen. Say it factually; you have verified it yourself.

ON SCREEN Back on the /health page

SAY: “Same code, same behaviour: locally, inside Docker, and on this public URL. All ten official cases reproduce the published optimum, and two hundred and twenty two automated tests pass. The repository and the live link are below. Thank you.”

DO Stop moving the mouse. Look at the camera if you can, otherwise look at the screen and speak to it like a person.

DELIVERY Land the last two sentences slowly. Do not add anything after “thank you” — stop recording.

What makes this project different — and when to say it

Ten features worth naming out loud. Do not list them all at once — each one is already placed in a beat above. This table is your checklist.

What makes this different	Where to say it	Why it earns marks
The model interprets, the solver decides	0:15 beat	The language model only returns structured directives; it never sees the schedule and never does arithmetic. Say this explicitly — it is the exact boundary the rubric rewards, and it pre-empts any suspicion that the AI guessed a plan.
Two-pass lexicographic solver	0:15 beat	Pass one minimises cost; pass two minimises battery cycling while holding that cost, so the plan has no pointless charge-and-discharge churn. Very few teams do this, and it is visible in the schedule.
Independent validator replays every plan	0:15 beat	Before a success response leaves the service, a separate module recomputes all twenty-four balances, every directive constraint and all three totals. A plan that fails is rejected rather than returned, so a cheap-but-invalid plan can never reach the judge.
Start-inclusive, end-exclusive windows	1:25 and 1:50 beats	Say the convention out loud. It matches the official sample data, and it is the kind of detail that separates a careful team from a lucky one.
No-op safety	0:58 and 1:50 beats	Unrelated notes must not become constraints. Showing a live no-op proves the interpreter is not hallucinating rules — a failure mode judges specifically look for.
Live generalisation, not phrase matching	1:50 beat	You run a note that is not in the sample set, with completely different wording, and get the same directive. Since the hidden tests are paraphrases, this is the single most valuable thing you can show on camera.
Graceful degradation	optional, 2:45 beat	If a set of directives is jointly infeasible, the service retries with a reduced directive set and reports the run as degraded instead of failing. It also paces itself on the provider's rate-limit headers, and if the model is unreachable it falls back deterministically and says so in the log rather than pretending.
Strict error taxonomy, no leakage	2:25 beat	400 for unparseable bodies, 422 for schema failures, 413 for oversized payloads, 405 and 404 for wrong routes. Error bodies carry a type and a message only — never a key, never a stack trace.
	2:25 beat	

Verified container, not a claimed one		The image was built, started, health-checked and exercised with a live model call inside the container. The image carries no credentials — that was confirmed by scanning the built image.
222 automated tests	2:45 beat	195 offline tests that need no key, plus 27 live tests that call the model. Among them are tests that build deliberately invalid plans and assert that the validator rejects them.

Numbers cheat sheet

Every figure below was measured against the live service. If a number on screen differs in the last decimals — for example 38365.000001 instead of 38365.00 — that is the solver's tie-break tolerance, within 0.000001 of the published optimum. Do not mention it unless a judge asks.

Check	Exact result	Timing / totals
GET /health	200 {"status":"ok"}	about 10 ms
SAMPLE-01 (official)	solar_reduction hours [12, 13] factor 0.25 + no_op	2692.50 kWh / 38365.00 BDT / peak 175.00
SAMPLE-05 (official)	max_grid_window hours [18, 19, 20] cap 155 kWh	2430.00 kWh / 33950.00 BDT / peak 175.00
Paraphrase of the same cap	max_grid_window hours [18, 19, 20] cap 155 kWh	2430.00 kWh / 33950.00 BDT / peak 175.00
Unrelated note	no_op, applies false, structured_adjustment null	2430.00 kWh / 33950.00 BDT / peak 175.00
Charging ban 2 AM to 5 AM	no_charge_window hours [2, 3, 4]	2430.00 kWh / 34130.00 BDT / peak 175.00
Bad route	405 {"detail":"Method Not Allowed"}	instant
Unknown path	404 {"detail":"Not Found"}	instant
Malformed body	400 {"error":{"type":"malformed_json", ...}}	instant
Cold model call	language model interpreting the note	0.3 to 1.8 s
Repeat of the same note	served from the interpretation cache	about 10 ms

If something goes wrong mid-recording

Symptom	What to do
A request is slow	Keep talking: “the model call is in flight — that is the language model reading the note, not a lookup table.” Never click Execute twice.
Response code is 422	The paste was edited or cut short. Clear the box, paste again from the companion guide, Execute. Cut that take in editing if you prefer.
Try it out button missing	You have not expanded the POST /optimize-energy row yet. Click the grey bar first.

A page will not load at all	Stop recording. Restart the tunnel, confirm the new link on /health, and start the take again — the URL changes every restart.
You stumble on a line	Pause, breathe, and repeat the sentence from its beginning. Cut the pause in editing; it reads as thoughtful, not as a mistake.

If a judge asks

Q. Is the AI doing the optimisation?

No. The language model only converts notes into structured directives. All scheduling and arithmetic happen in a deterministic linear program, so the same scenario and the same notes always produce the same plan.

Q. What if the model returns something nonsense?

Guardrails normalise it before it can reach the solver: hours are snapped to valid slots, factors and capacities are clamped to physical range, unknown directive types collapse to no_op, and exactly one entry per note is enforced.

Q. What if the notes contradict the physics?

If the combined constraint set is infeasible, the service retries with a reduced directive set and marks the response as degraded, rather than returning an invalid schedule or an error.

Q. Why this model?

Groq's gpt-oss-20b, an open-weight model. It was the fastest option that returned correct structured output in our own benchmark, which matters because every request needs a live model call.

Q. How fast is it, and does it fit the limits?

A cold request is typically under two seconds end to end; a repeated note is about ten milliseconds from cache. Well inside the thirty-second timeout and the five-second p95 target.

Q. Is the URL permanent?

It is a live tunnel in front of the running service. The repository ships a Dockerfile and deployment instructions so it can be placed on a permanent host.

Q. How did you avoid leaking secrets?

The key lives only in a local .env file that is excluded from git, the built image was scanned and confirmed to contain no credentials, and error responses never echo request internals.

Closing rule. Say only what you can point at on screen. Everything in this script is verified: the ten official optima, the paraphrase results, the error codes, the Docker run and the test count. If you are asked about something you did not verify, say so plainly — judges reward precision far more than confidence.